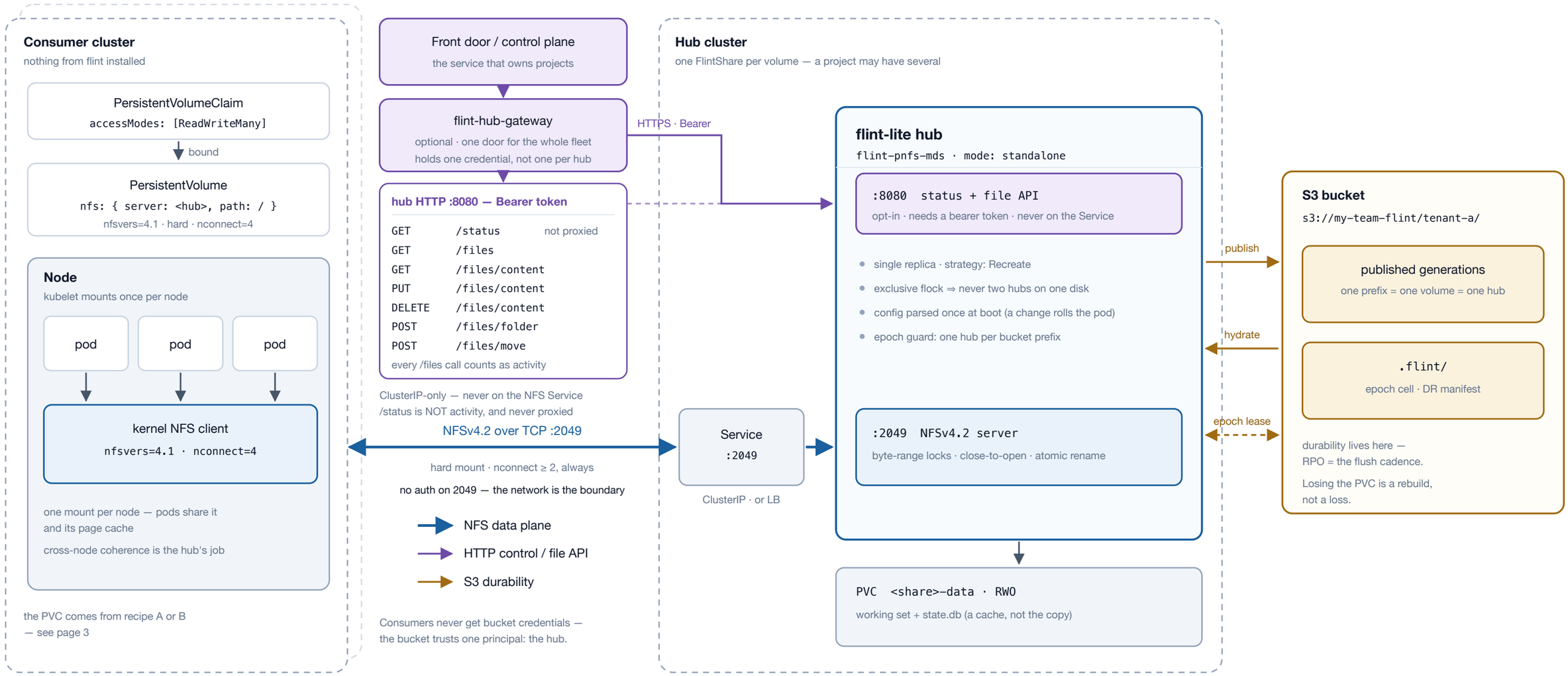
Flint-lite — one hub, one volume

A single pod serves real POSIX over NFSv4.2. The PVC is the working set, an S3 prefix is the durability, and a second listener exposes the same filesystem over HTTP for callers that cannot mount — reached through one gateway rather than one address per hub. One hub serves exactly one prefix; a project that needs several volumes has several hubs.



One hub is the coherence authority for one tree — two hubs over one prefix is split-brain, which the store-side epoch lease fences and the operator refuses up front. **nconnect ≥ 2 is not optional:** without it the kernel opens exactly one connection and silently refuses every additional trunk — no error, on either side, ever. **The two doors are deliberately separate:** :2049 is on the Service and may become a LoadBalancer; :8080 can rewrite every file in the share, so it stays ClusterIP-only and never joins it. Each HTTP endpoint is an NFS compound dispatched in-process, not a second reader of the directory — so it inherits eviction handling, the write gate, symlink rules and the tier's capture notes. A cold file read through either door hydrates from the bucket first: NFS parks the caller with `NFS4ERR_DELAY`, HTTP answers `503 + Retry-After`. **They are secured differently, and only one of them has a credential.** The file API needs a bearer token per share and mounts no routes at all without one — `404`, not `401`. Port 2049 is AUTH_SYS: the client asserts its own uid and the server takes it, so POSIX modes are enforced against a self-declared identity and **reachability is the boundary**. That control is `networkPolicy` in both charts — off by default, admitting nothing when both client lists are empty, and silently ignored by a CNI that does not enforce it.

The flint-lite operator — a fleet of shares

One custom resource replaces one helm release, per hub. The operator re-renders the same four objects from the CR on every pass — there is no reusable release state to go stale, and nothing it does ever reaches the bucket.

FlintShare — one per volume

flint.io/v1alpha1 · namespaced

```
metadata:
  name: fs-tenant-a
  labels:
    flint.io/project-id: tenant-a
  annotations:
    flint.io/requested-at: <now>
spec:
  bucket: my-team-flint
  keyPrefix: tenant-a/    # immutable
  credentialsSecretRef: flint-s3
  persistence: { size: 20Gi }
  monitoring:
    enabled: true
    fileApi: { enabled: true }
  idle:
    suspendAfterSecs: 900
    hibernateAfterSecs: 86400
  settings:               # typed
    hydrateWarmAfterImport: true
```

a typo in `settings` is refused at admission — the knobs are schema, not a free-form map

bucket and keyPrefix are immutable once set

watch

flint-lite-operator

namespace flint-system · Deployment
2 replicas, leader-elected

1. **arbitrate** — at most one share per bucket subtree
2. **render** — four objects from the CR + defaults
3. **repair drift** — re-renders every pass; no release state
4. **run the idle ladder** — polls the hub's /status, then decides
5. **publish status** — phase · address · serverId · conditions

liveness is the leader Lease's renewTime — a share cannot wake while nothing reconciles

reconcile concurrency is capped, deliberately

apply

renders exactly four objects

the same four the lite chart renders

ConfigMap
`mds.yaml`
hashed into checksum/config

PersistentVolumeClaim
`<share>-data · RWO · 20Gi`
no ownerRef — never GC'd

Service
`:2049`
ClusterIP · NodePort · LoadBalancer

Deployment
`replicas: 1 · strategy: Recreate`
the hub pod itself

a golden test fails the build if chart and operator ever diverge

runs

GET /status — is it idle? is it clean?

S3 bucket prefix

`s3://my-team-flint/tenant-a/`

- **published generations** — durability + DR manifest
- **`.flint/`** — epoch cell — one hub per prefix

the operator never touches the bucket

publish

hydrate

flint-lite hub

one pod · one volume

- `:2049 NFSv4.2` — on the Service
- `:8080 status + files` — the operator polls this

Three things it will not do

1 · It never touches a bucket. Deleting a share deletes Kubernetes objects; S3 is exactly as durable as before.

2 · It never garbage-collects a PVC — the claim carries no ownerReference, deliberately.

3 · It will not run two hubs on one bucket subtree.

The fleet is one command

```
$ kubectl get flintshares -A

NAMESPACE    NAME       PHASE          ADDRESS
workspaces   tenant-a   Ready          tenant-a.workspaces.svc:2049
workspaces   tenant-b   Starting
workspaces   tenant-c   IdleSuspended
```

The front-door contract

```
GET   flintshares/fs-<id> → 404? create
PATCH flint.io/requested-at: <now>
poll  until .status.phase == Ready
read  .status.address, .status.serverId
```

a changed serverId means a fresh PVC — remount

with a gateway in front, the middle three are one POST …/wake

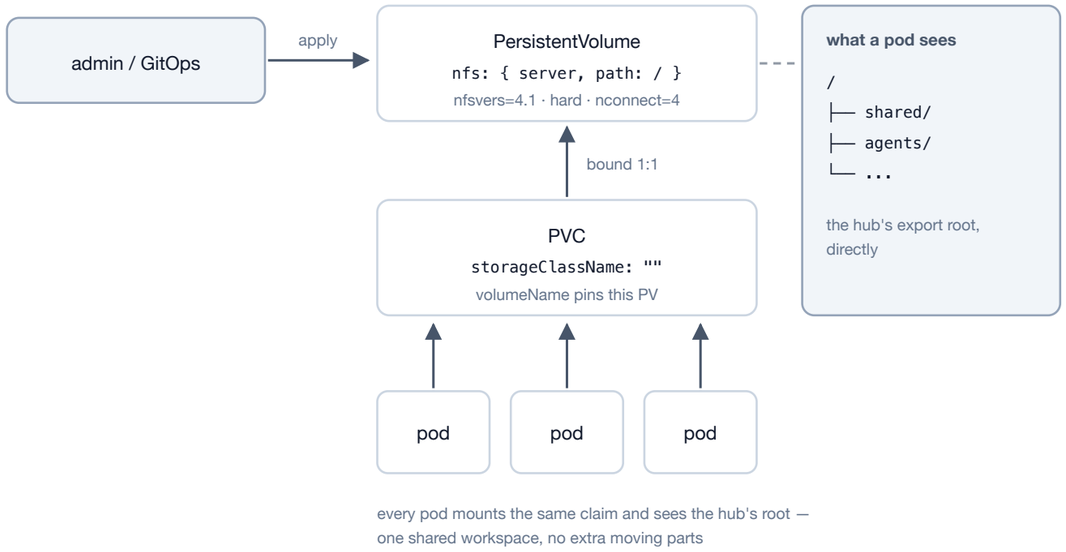
Label the share `flint.io/project-id`, and `flint.io/volume-id` too once a project owns more than one — those labels are what the gateway resolves `/v1/projects/<id>/volumes/<v>` against, and `volume-id` defaults to the CR name. Deriving the name from the project id (`fs-<project-id>`) still works for a single-volume project and is what makes ensure-live idempotent: two front-door replicas racing a first touch issue the same create, one gets `409 AlreadyExists`, and both proceed. **Uniqueness is enforced from the controller's cache**, not by the API server: Kubernetes cannot express "at most one share per `(endpoint, bucket, prefix subtree)`" in CEL, since both CEL and a ValidatingAdmissionPolicy see one object. Oldest wins; the loser goes `Failed` with a `Conflict` condition and gets no hub. **The project id is not part of that key**, and nothing in the operator reads it — which is precisely why one project may own several volumes: they collide only if their prefixes do. **That enforcement stops at the cluster boundary.** Two clusters each admit a share on one prefix and neither sees the other — and the store-side epoch only catches it when the prefixes are *equal*, because `tenant-a/` and `tenant-a/sub/` mint different epoch objects and never contend. Own prefix allocation upstream of Kubernetes.

Giving client pods a PVC — two recipes

Consumers install nothing from flint either way: the data path is the node kernel's NFS client. The choice is only who mints the PV — you, once, by hand; or a stock provisioner, once per workspace.

Recipe A — static PV, zero install

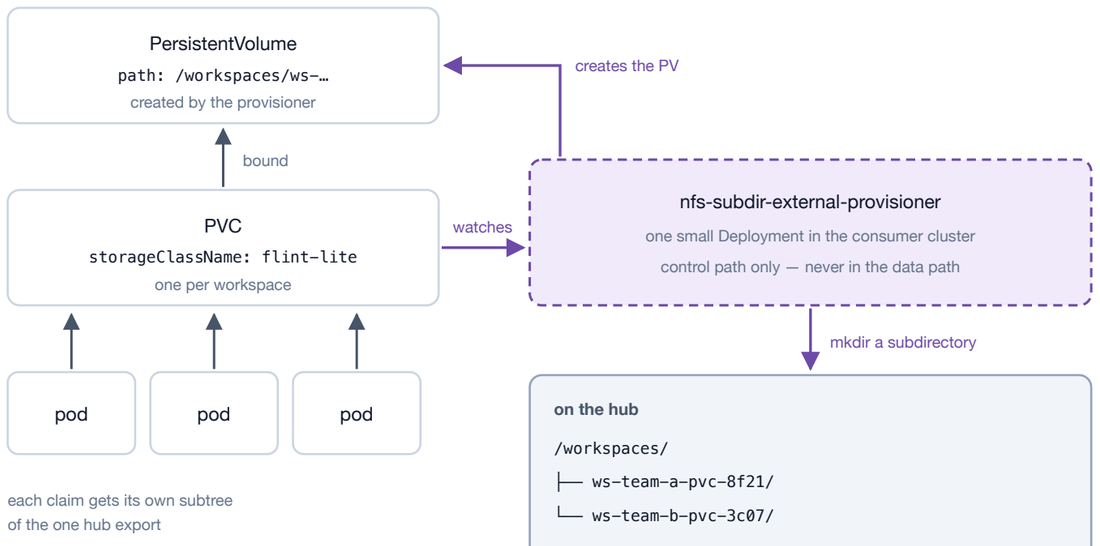
one PV pinned to the hub's root; every pod shares one namespace



every pod mounts the same claim and sees the hub's root — one shared workspace, no extra moving parts

Recipe B — a PVC per workspace

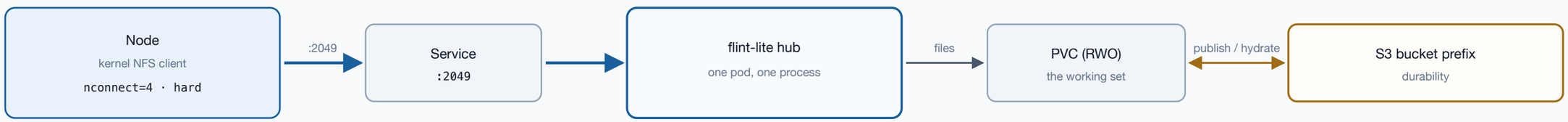
the stock nfs-subdir-external-provisioner mints a fresh subdirectory per claim



each claim gets its own subtree of the one hub export

Both recipes end in the same data path

the provisioner mints directories and PVs; it never carries a byte



The same `nconnect` caveat applies to both recipes — recipe B passes mount options straight to the node's mount, so leaving it out gets one connection per node and no error saying so. **Recipe A is one shared tree;** recipe B is StorageClass semantics on top of it, so pick B when each workspace should be isolated and minted on demand, A when the point is that everybody sees the same files. Kubelet mounts once per node: pods co-scheduled on a node share one kernel client and one page cache, while cross-node and cross-cluster coherence — locks, close-to-open — is the hub's job. **Either way, a suspended hub is not woken by a mount.** A hard mount against a scaled-to-zero share hangs, because there are no endpoints and nothing to notice the attempt — which is why the wake annotation is written before an address is handed to anyone.

Starting, winding down, waking

A tiered hub does real work before its listener binds, and the operator winds an unused share down in two rungs — each opt-in, each giving back something different. Nothing in the data path ever starts a hub.

Starting a hub — what happens before port 2049 opens

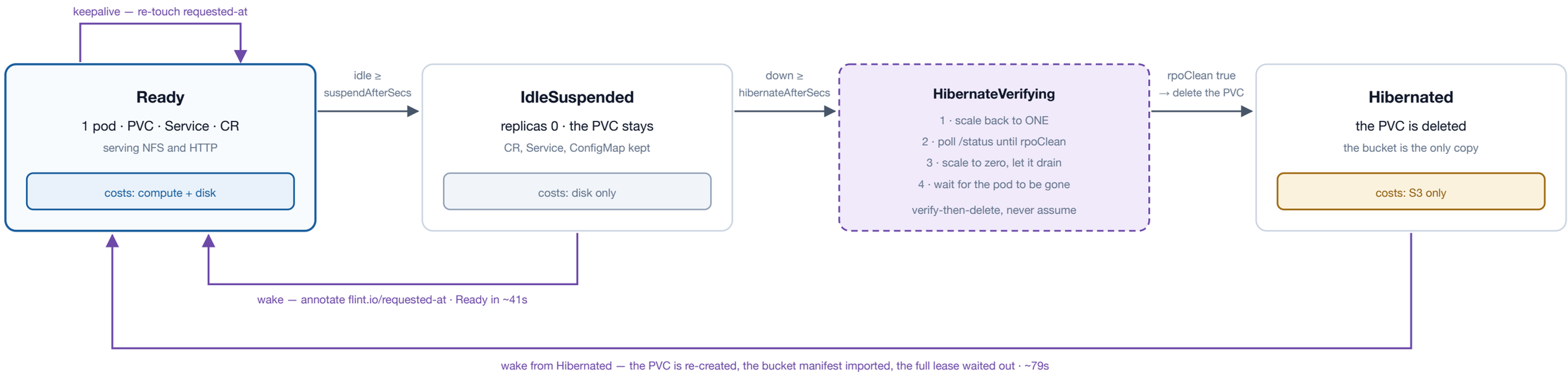
A pod in Running / not-Ready is working, not stuck.

The startupProbe budgets 60 × 10s for all of it.

Killing a Starting hub kills a takeover or an import.

Poll /status: it tells importing from wedged.

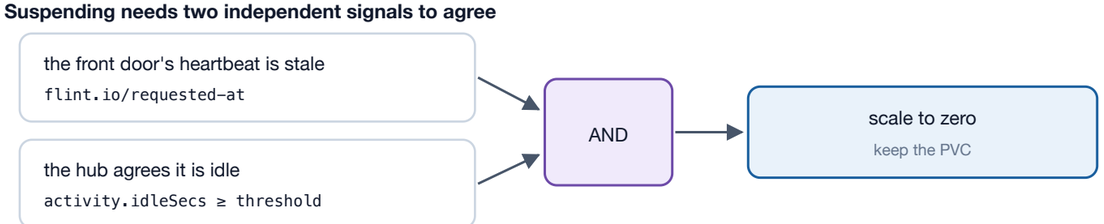
The idle ladder — each rung gives back something different



Nothing in the data path wakes a share: an NFS mount against a scaled-to-zero hub hangs, the file API answers 503 fast. Something must write the annotation — and a share cannot wake while no operator is reconciling.

spec.lifecycle: Suspended is an admin decision and a wake request does NOT override it — the phase reads Suspended, not IdleSuspended, so a front door can tell "will wake on request" from "never coming back".

Suspending needs two independent signals to agree



A hub that cannot be polled is never suspended — an unreachable hub is an unknown hub, not an idle one, and HubReachable says so.

What each rung still holds

● held ○ released

	CR	Svc + CM	PVC	Pod
Ready	●	●	●	●
IdleSuspended	●	●	●	○
Hibernated	●	●	○	○

Deleting the CR drops every object — an archived project costs zero Kubernetes objects. The bucket is never touched, under any policy.

The two rungs are not one setting with two numbers. Suspend keeps the disk, so waking is a pod start and an epoch re-claim on the same `state.db` — safe for any share, tiered or not. Hibernate deletes the PVC, so it requires `spec.bucket` and waking is a full DR import. **Hibernation is verified at drain time, not assumed:** the hub exits 0 whether or not it flushed and scale-to-zero destroys the pod, so the operator scales back to one and waits for `rpoClean` before it deletes anything. `rpoClean: null` — a share with no bucket — is a refusal, not a pass. **Hibernated wakes are slower than they look** (~79s against ~13s for the same-identity case): destroying the volume destroys the `serverId`, so self-recognition cannot short-circuit the lease and the claim waits out the full `lease_misses × heartbeat`. Every long-parked project pays that on its first open. **Do not keepalive from a liveness poller.** `/status` is deliberately not activity and the file API deliberately is — a UI that refreshes a listing on a timer pins its project awake forever, and a conditional GET answering `304` pins it exactly as hard.